AWS Amplify 집중 조사 보고서

얼마였지의 모바일 백엔드 구현 방법과 조건

과목	고급모바일프로그래밍
팀 구성원	2191298 김종현 · 2271494 이재영 · 2311122 신지현
문서 정보	AWS Amplify 조사 v1.0 · 2026년 10월 9일
조사 대상	Amplify Auth·Data·Functions와 연결되는 AWS 자원

조사 목적

얼마였지의 사용자별 장보기 목록과 구매 가격 기록을 AWS Amplify로 구현하는 방법을 조사한다. 사용자 구분, 기기 간 동기화, 구매 확정, 가격 조회·공유를 요구사항과 연결하고 기본 지원과 추가 개발 조건을 정리한다. AWS Amplify의 채택을 확정하는 문서는 아니다.

조사 범위

핵심 기능	요구사항	확인할 내용
사용자와 목록 동기화	F-08~F-12	본인 접근, 오프라인 대기열, 충돌·삭제·중복
구매 확정	F-20~F-22	부분 구매, 목록·가격의 원자적 반영, 실패 복구
가격 조회와 공유	F-23~F-28	매장별 최근 기록, 비교 조건, 공개 필드·공유 해제

조사의 핵심 전제

Gen 1 DataStore의 자동 동기화와 Gen 2 API 구성을 구분한다. Gen 2 기능표는 DataStore와 그 충돌 감지를 지원하지 않는 것으로 구분하므로, Gen 2의 오프라인 동기화는 별도 구성으로 분석한다. Gen 1의 기능을 Gen 2의 기본 기능처럼 설명하지 않는다. [2] [3]

기록은 기기에 먼저 저장하고 서버 실패 시 보존한다. 원본 사진은 서버에 전송하지 않으며 사용자가 검토한 텍스트만 저장한다. 최종 로그인 방식과 공개·충돌 정책은 팀의 확정 대상이다.

기준 자료: 모바일 백엔드 서비스 강의 자료, 프로젝트 제안서, 요구사항 분석서 v1.0, UI 프로토타입 설계서 v1.0. 예제는 조사용 코드이며 실행·배포 증빙이 아니다.

1 서비스 구성과 특장점

AWS Amplify는 인증·데이터 API·서버 함수를 정의하고 앱과 연결하는 개발 도구와 라이브러리를 제공한다. Gen 2에서는 TypeScript로 백엔드 자원을 정의하고 Flutter 클라이언트는 생성한 설정과 모델을 이용해 API에 접근한다. [1]

구성 요소	주요 역할	얼마였지에서의 역할
Amplify Auth / Cognito	사용자 인증과 세션	같은 사용자 확인, 개인 자료 구분
Amplify Data / AppSync	GraphQL query·mutation·subscription	목록·구매 반영과 가격 조회 API
DynamoDB	항목 저장, 조건부 쓰기와 트랜잭션	버전 검증, 요청 ID, 목록·가격 저장
Amplify Functions / Lambda	사용자 정의 서버 로직	충돌 판정, 구매 확정, 공개 가격 관리

구성 근거 [7] [10] [11] [13]

기능 연결 구성안

> Flutter 앱 → Cognito 인증 → AppSync API → 서버 처리 → DynamoDB

앱은 로컬 목록·가격 기록·전송 대기열을 관리한다. 서버는 요청자와 소유자를 확인하고 항목 버전·구매 상태·공개 정책을 검증한다. Lambda를 사용하는 처리와 AppSync resolver에서 직접 처리하는 방식은 기능에 따라 선택할 수 있다. [10]

특장점과 개발 부담

특장점: 인증·API·함수를 코드로 함께 관리하고 AWS 자원을 연결할 수 있다. 사용자 정의 mutation과 DB 트랜잭션을 이용해 구매의 여러 변경을 서버에서 묶는 구성이 가능하다.

개발 부담: 모델·인증 규칙뿐 아니라 함수 권한, 인덱스, 배포 환경과 비용을 관리해야 한다. 자동 생성 CRUD만으로 프로젝트의 충돌 선택·구매 일관성·장기 재전송까지 보장되지는 않는다.

Hosting은 웹 배포 기능이다. 모바일 앱의 백엔드 이용에 웹사이트 Hosting을 필수로 포함할 필요는 없으며, 백엔드는 실제 구성한 서비스별로 과금된다. [21]

2 Gen 1과 Gen 2의 지원 차이

항목	Gen 1 DataStore 구성	Gen 2 API 구성
백엔드 정의	CLI와 GraphQL 모델 중심	TypeScript 자원 정의
기기 저장	DataStore가 모델 저장소 제공	앱의 로컬 저장 계층 별도 구성
오프라인 전송	변경 대기·재전송과 동기화 제공	앱의 대기열과 재연결 반영 별도 구성
충돌	버전 추적과 Auto Merge 등 전략	기준 버전 조건·충돌 응답 직접 설계
실시간	observe·observeQuery와 동기화	AppSync subscription과 재조회
구매 원자성	여러 save 호출은 구매 전체 트랜잭션과 구분	사용자 정의 mutation과 DB 트랜잭션 설계

공식 기능 구분 [2] [3] [4] [5] [18]

Gen 1의 기본 충돌 정책

DataStore의 기본 Auto Merge는 단일 필드에서 서버 값을 우선하는 정책이다. Optimistic Concurrency와 Custom Lambda 전략, 클라이언트 conflictHandler 등도 제공한다. F-11의 '두 변경을 보존하고 사용자 선택'은 기본 자동 병합과 다르다. [5]

사용자 선택형으로 구성하려면 충돌 후보를 별도 보관하고 선택 화면과 재반영을 연결한다. DataStore의 로컬 저장 완료와 다른 기기의 반영 완료도 구분하며 outbox 등 이벤트를 통해 상태를 관찰한다. 미해결 충돌이 남으면 동기화 완료로 표시하지 않는다. [6]

이 조사에서 예제를 읽는 기준

이후의 TypeScript 모델·API와 서버 구성안은 Gen 2 중심으로 설명한다. Gen 1 DataStore는 자동 동기화의 지원 범위를 이해하는 자료로 구분한다. SDK 버전 번호의 v1·v2와 백엔드 세대 Gen 1·Gen 2는 같은 구분이 아니다.

공식 이전 가이드는 DataStore 대신 API와 별도 로컬 캐시·오프라인 계층을 조합하는 방법을 설명한다. 기능 차이만으로 특정 세대의 채택이나 지원 종료일을 단정하지 않는다. [3]

3 사용자 확인과 접근 통제

F-08과 C-06은 본인 목록만 접근하고 계정 전환 시 미동기화 변경을 다른 계정과 합치지 않도록 요구한다. Cognito 인증과 데이터 소유자 권한을 연결할 수 있다. 로그인 방식은 D-02에서 확정하며 아래 이메일 로그인은 가능한 구성의 예시다. [7] [8]

```
// amplify/auth/resource.ts — Gen 2 예시
import { defineAuth } from '@aws-amplify/backend';
export const auth = defineAuth({
  loginWith: { email: true },
});
```

```
// a.schema 안의 개인 목록 모델 예시
ShoppingItem: a.model({
  title: a.string().required(),
  quantity: a.integer().required(),
  completed: a.boolean().required(),
  version: a.integer().required(),
  deleted: a.boolean().required(),
})
  .disableOperations(['create', 'update', 'delete'])
  .authorization(allow => [allow.owner().to(['read'])]),
```

schema 정의·defineData·함수 연결은 생략했다. owner 규칙은 소유자 필드를 이용한다. 수정은 이후 설명하는 사용자 정의 API로 제한하는 구성안이다. 자동 생성 읽기 API도 본인 범위로 제한하고, 서버 저장 시 소유자 형식을 인증 규칙과 일치시킨다. [8] [9]

함수에서 다시 검사할 사항

allow.authenticated()는 로그인한 호출자라는 조건이며 특정 항목의 소유권 검사를 대신하지 않는다. 서버 함수는 AppSync가 검증한 요청자 정보로 계정을 결정하고 입력의 owner 값을 신뢰하지 않는다. 직접 DynamoDB에 쓰면 모델의 자동 권한·검증 로직을 거치는 것으로 가정하지 않는다.

함수의 AWS 실행 권한과 앱 사용자의 데이터 권한은 다르다. 필요한 테이블·작업만 함수에 허용하고, 소유권·입력 범위·버전은 함수 로직과 쓰기 조건에서 검증한다. [12]

계정 전환

전환 시 이전 계정의 전송·구독을 중단하고 응답 반영을 차단한다. 로컬 대기열은 사용자별로 보존한다. SDK 로그아웃을 호출했다고 앱의 로컬 데이터까지 올바르게 분리되는 것은 아니다. [20]

4 목록 동기화 API와 변경 대기열

핵심 기능 1: F-09~F-12의 추가·수정·삭제·완료 상태 반영은 Gen 2에서 로컬 대기열과 서버 API로 구성한다. 예시의 updateShoppingItem은 조사에서 정의한 사용자 API 이름이며 SDK의 기본 함수가 아니다.

```
// a.schema 안의 변경 API 예시 — Gen 2
ChangeResult: a.customType({
  status: a.string().required(),
  version: a.integer(),
}),
updateShoppingItem: a.mutation()
  .arguments({
    requestId: a.string().required(),
    itemId: a.id().required(),
    expectedVersion: a.integer().required(),
    quantity: a.integer().required(),
  })
  .returns(a.ref('ChangeResult'))
  .authorization(allow => [allow.authenticated()])
  .handler(a.handler.function(updateItem)),
```

updateItem은 defineFunction으로 정의한 Lambda 자원이다. import·함수 구현·권한 설정을 생략했다. 실제 충돌 응답에는 본인에게 허용되는 서버 후보 값과 삭제 여부도 포함해야 한다. 로그인 조건 외에 소유권·수량·기준 버전 검사는 서버에 추가한다. [10] [11]

서버 반영 순서 구성안

> 인증·입력 확인 → 동일 요청 결과 확인 → 기준 버전 검증 → 변경과 요청 결과를 함께 저장

로컬 요청에는 계정·요청 ID·항목 ID·기준 버전·변경 내용을 보관한다. 서버 버전이 기준과 다르면 로컬·서버 두 후보를 보존하고 충돌로 반환한다. 일치하면 버전을 증가시키며 반영한다. 사용자 계정과 버전 조건을 쓰기 시점에도 확인한다.

동기화 중 새 변경

전송 중인 요청을 고정하고 이후 변경은 다음 요청으로 보관한다. 이전 요청이 성공했다고 새 변경까지 대기열에서 지우지 않는다. 실패하거나 응답이 유실되면 같은 ID·내용으로 재시도한다. 단순 타임스탬프 덮어쓰기로 충돌을 조용히 해결하지 않는다.

5 충돌과 재연결 실패 처리

상황	처리 구성안	보존할 내용
두 기기 같은 항목 수정	서버와 기준 버전 대조 후 사용자 선택	양쪽 값·기준 버전·미해결 상태
한 기기 수정, 다른 기기 삭제	삭제 후보와 수정 후보를 함께 제시	삭제 버전과 수정 요청
서버 저장 후 응답 유실	같은 요청 ID의 기존 결과 반환	요청 ID·내용 해시·저장 결과
재접속 또는 구독 재연결	서버 기록과 삭제 표시를 재조회	미전송 변경을 별도로 보존
선택 중 서버가 다시 변경	선택 시점 버전을 다시 확인	새 충돌 후보와 이전 선택 정보

요청 중복 방지의 기간

DynamoDB TransactWriteItems의 ClientRequestToken은 완료 후 10분의 유효 기간을 가진다. 오프라인 상태가 오래 지속되는 앱의 중복 방지를 이 토큰만으로 보장할 수 없다. 사용자와 요청 ID를 키로 한 처리 결과를 서버에 저장하고 재시도 정책에 맞는 기간 동안 보관하는 구성이 필요하다. [13]

동일 ID에 다른 요청 내용을 보내면 오류로 처리한다. 동시에 같은 요청이 들어와도 처리 결과의 신규 생성 조건과 변경을 한 트랜잭션으로 묶어 하나만 반영한다. 결과 보관 기간과 만료 후 재전송 처리는 별도 운영 정책이다.

실시간 알림과 확정 데이터

subscription은 온라인 변경 알림의 수단이다. 끊긴 동안의 모든 변경과 삭제를 앱 대기열처럼 보관하는 것으로 가정하지 않는다. 재연결하면 페이지네이션을 포함해 서버 상태를 재조회하고, 미전송 로컬 변경과 버전별로 조정한다. [18]

Lambda가 DynamoDB에 직접 저장하는 구성에서는 모델의 기본 구독에 자동 알림이 발생한다고 가정하지 않는다. 사용자 정의 mutation에 연결한 구독·변경 알림을 구성하거나 명시적 재조회로 반영한다. 통신 복구 후 알림 없이도 최종 상태가 일치하는지 확인한다. [19]

사용자 정의 subscription의 authorization은 owner 전략을 지원하지 않는다. 인증된 사용자 전체가 개인 목록 변경을 받지 않도록 서버가 검증한 계정에 맞춰 구독 필터·권한을 구성해야 한다. 클라이언트가 전달한 계정 필터만 믿지 않으며 다른 계정 구독 시도를 시험한다. [19]

삭제 표시를 너무 일찍 지우면 오래된 기기가 삭제 사실을 알 수 없다. 전체 재조회와 삭제 표식 정리 정책은 장기 오프라인 복귀 시험까지 포함해 확정한다.

6 구매 확정의 원자적 저장

핵심 기능 2: 우유 3개 중 2개를 구매하면 잔여 1개·미완료, 구매분 2개·완료, 가격 기록을 함께 남긴다. 단일 confirmPurchase mutation에서 목록 버전·소유권·구매 입력을 검증하고 DB 트랜잭션으로 반영하는 구성이다.

```
// Lambda의 저장 부분을 설명하는 TypeScript 코드 조각
await database.send(new TransactWriteCommand({
  TransactItems: [
    { Update: {
      TableName: itemsTable, Key: originalItemKey,
      UpdateExpression: 'SET quantity = :left, version = :next',
      ConditionExpression:
        '#owner = :user AND version = :base AND deleted = :no',
      ExpressionAttributeNames: { '#owner': 'owner' },
      ExpressionAttributeValues: validatedValues,
    } },
    { Put: {
      TableName: itemsTable, Item: completedItem,
      ConditionExpression: 'attribute_not_exists(id)',
    } },
    { Put: {
      TableName: pricesTable, Item: priceRecord,
      ConditionExpression: 'attribute_not_exists(id)',
    } },
    { Put: {
      TableName: requestsTable, Item: requestResult,
      ConditionExpression: 'attribute_not_exists(id)',
    } },
  ],
}));
```

@aws-sdk/lib-dynamodb의 TransactWriteCommand와 DocumentClient 초기화를 생략했다. id가 기본 키인 테이블을 가정한 부분 구매 예시이며 모든 객체·조건값은 서버 검증 후 준비한다. 소유자·버전·잔여 수량·안정적인 ID·요청 내용 해시를 포함해야 한다. 트랜잭션 원자성의 근거는 DynamoDB 문서다. [13] [14]

예시를 확장할 조건

전체 구매이면 잔여 0개 항목을 만들지 않고 원래 항목을 완료 처리한다. 초과 구매·목록 연결 없음·여러 상품은 별도 입력 검증을 적용한다. 동일 항목을 한 트랜잭션에서 여러 동작으로 중복 지정할 수 없으므로 변경을 항목별로 정리한다.

응답 유실 후 재시도에서는 먼저 저장된 요청 결과를 조회한다. 동시 실행으로 트랜잭션이 취소되면 기존 요청 결과와 충돌 여부를 다시 확인한다. 서버 원자성과 기기의 로컬 저장은 별개이며, 기기는 서버 확인 전까지 동기화 대기로 유지한다.

7 매장별 최근 가격 조회

핵심 기능 3: F-23~F-26은 본인 이력과 다른 사용자의 같은 조건 구매 가격을 조회한다. '최근'은 구매일을 우선하고, 같은 구매일이면 등록 시점이 늦은 기록을 선택한다.

조회 키와 인덱스 구성안

comparisonStoreKey는 상품·규격·단위·묶음·할인 조건과 매장을 조합한 키다. purchasedOrder는 구매일·서버 등록 시점을 같은 길이의 정렬 가능한 문자열로 구성한다. 공유 조회의 최종 응답은 허용된 필드만 반환한다.

```
// a.schema 안의 공개 가격 인덱스 구조 예시
SharedPrice: a.model({
  comparisonStoreKey: a.string().required(),
  purchasedOrder: a.string().required(),
  productName: a.string().required(),
  storeName: a.string().required(),
  purchasedOn: a.date().required(),
  quantity: a.integer().required(),
  paidAmount: a.integer().required(),
})
  .secondaryIndexes(index => [
    index('comparisonStoreKey').sortKeys(['purchasedOrder']),
  ])
  .disableOperations(['mutations', 'queries', 'subscriptions'])
  .authorization(allow => [allow.owner()]),
```

개인 데이터와 공개 응답을 분리하는 인덱스 예시다. owner는 내부 소유자 필드다. 규격·할인 필드 등은 생략했다. 자동 생성 API를 비활성화하고 별도의 조회 함수에만 테이블 읽기 권한을 주는 구성으로 설명한다. 함수 연결·리소스 권한·공유자와 원본 기록 연결은 별도로 구현해야 한다. [9] [12] [15]

서버 조회 흐름

> 비교 조건 검증 → 매장별 인덱스 조회 → 최신 후보 확인 → 공개 상태 재확인 → 응답

서버는 정렬 키 내림차순으로 후보를 조회한다. 최신 후보가 공유 해제되었으면 다음 후보를 조회하고 페이지 토큰을 처리한다. 매장이 여러 곳이면 매장별 결과를 묶는다. 다른 상품·규격·할인 조건은 같은 조회 키에 포함하지 않는다.

본인 이력은 개인 구매 모델에서 소유자와 구매일을 기준으로 조회한다. 기록 없음·조회 실패·비교 불가를 구분하고 현재 판매가·현재 최저가라고 표시하지 않는다.

8 가격 공개와 공유 해제

개인 모델과 공개 응답 분리

owner 권한 모델에는 사용자 식별 정보가 포함될 수 있다. 다른 사용자에게 모델 전체를 공개하지 않고 공개용 응답을 정의한다. 공개 ID·상품·매장·구매일·규격·수량·금액·할인 조건만 반환하고 내부 owner·요청 ID·개인 목록 연결은 제외한다. [8]

동작	서버 처리 구성안	개인 기록 영향
공유 시작	본인 가격 확인 후 공개 사본과 연결 저장	개인 이력 유지
공유 해제	공개 상태 변경과 공개 사본 제거	개인 이력 유지
가격 정정	개인 기록과 공개 사본 조건·인덱스 갱신	정정 결과 반영
가격 삭제	관련 공개 사본과 내부 연결 정리	확정한 삭제 정책 적용

인덱스 반영 지연

DynamoDB의 글로벌 보조 인덱스는 원본 테이블 변경을 비동기로 반영하고 최종 일관성을 사용한다. 공유 해제 직후 이전 후보가 인덱스에 남을 수 있다. 인덱스 조회 결과를 그대로 반환하지 않고 원본 공개 상태를 다시 확인하는 구성과 지연 시험이 필요하다. [16]

공개 상태의 기준은 서버가 관리하고, 후보 원본은 기본 테이블의 강한 일관성 읽기로 확인한다. 공유 해제된 최신 기록을 제외했다면 다음 후보를 탐색한다. 네트워크 지연·캐시·조회 중 공유 해제 상황도 시험하고 새로 조회한 결과에 더 이상 노출되지 않는지 확인한다. [26]

동일 조건 비교를 위한 값

상품별 실제 결제 금액과 구매 개수를 저장한다. 영수증 합계를 임의로 상품 가격에 나누지 않는다. 용량·할인 조건을 모르는 기록과 배분 불가 쿠폰은 참고용으로 구분한다. 상품명을 비슷하게 인식했다는 이유로 동일 상품 키를 자동 확정하지 않는다.

공개 선택·해제·정정·삭제 정책은 D-05의 확정 대상이다. S3 원본 사진 업로드는 현재 요구사항의 저장·공유 구성에 포함하지 않는다.

9 Flutter 연결과 조사용 구축 절차

Gen 2의 공식 Flutter 시작 절차는 백엔드 생성, cloud sandbox 배포, 설정·모델 생성, Amplify 플러그인 초기화로 이어진다. sandbox는 AWS 클라우드 자원을 사용하는 개발 환경이므로 순수한 로컬 에뮬레이터와 구분한다. [1]

```
# 아래 명령은 조사용 절차이며 이번 작업에서 실행하지 않았다.
npm create amplify@latest -y
npx ampx sandbox --outputs-format dart --outputs-out-dir lib
npx ampx generate graphql-client-code --format modelgen   --model-target dart --out lib/models
```

앱에는 amplify_flutter, amplify_auth_cognito, amplify_api 등을 추가하고 생성된 출력 설정과 ModelProvider를 연결한다. SDK 버전·최소 플랫폼 조건은 실제 사용할 버전을 고정한 뒤 확인한다. 인증은 userPools 모드로 요청하고 비밀번호·토큰을 로그에 남기지 않는다. [17]

사용자 정의 변경 요청 호출 예시

```
// document는 서버 스키마와 일치하는 GraphQL mutation 문자열이다.
final request = GraphQLRequest<String>(
  document: updateItemDocument,
  variables: {
    'requestId': savedRequestId,
    'itemId': itemId,
    'expectedVersion': baseVersion,
    'quantity': validatedQuantity,
  },
  authorizationMode: APIAuthorizationType.userPools,
);
final response = await Amplify.API.mutate(request: request).response;
if (response.errors.isNotEmpty || response.data == null) {
  throw StateError('요청 결과 확인 필요');
}
```

초기화·import·GraphQL 문서를 생략한 Dart 코드 조각이다. response.data 안의 처리 상태·충돌 후보를 파싱해야 하며 위 코드만으로 성공을 확정하지 않는다. APIException·타임아웃·중복 실행·계정 변경·화면 종료를 처리하고 오류 시 로컬 요청을 유지한다.

호출 전 로컬 저장, 호출 후 현재 계정과 요청 세대 확인을 수행한다. 서버 확정 결과와 로컬 반영이 완료된 요청만 대기열에서 제거한다.

10 비용과 운영 조건

Amplify 백엔드는 구성한 AWS 서비스별 사용량이 비용을 만든다. 무료 할당 여부는 계정·지역· 요금제·사용량 조건에 따라 확인하며, Hosting 가격을 모바일 백엔드 전체 비용으로 대신하지 않는다. [21]

서비스	주요 비용 요소	얼마였지의 사용량 요인
Cognito	활성 사용자·요금제·인증 방식	사용자 수, 선택한 인증 기능
AppSync	query · mutation, 실시간 메시지·연결	동기화 요청, 가격 조회, 기기별 구독
DynamoDB	읽기·쓰기, 트랜잭션, 저장·인덱스	구매 묶음, 요청 결과 보관, 공개 사본
Lambda	호출 수·실행 시간·메모리	버전 검증, 구매 확정, 공개 가격 조회

공식 요금 근거 [22] [23] [24] [25]

견적을 위한 공통 입력

월 사용자·사용자당 기기·하루 목록 변경·구매 건수·가격 조회 횟수를 정한다. 구매 1건당 변경되는 항목·기록 수, 인덱스 수, 재시도율, 함수 실행 조건을 더해 산정한다. 이번 조사에서는 사용량과 리전을 확정하지 않았으므로 월 비용을 제시하지 않는다.

운영에서 확인할 조건

요청 결과·삭제 표식의 보관 기간, 서버 로그의 개인정보 제외, 함수의 테이블 권한, 환경별 설정·백업·데이터 이전 방법을 기록한다. 모델과 인덱스를 변경할 때 기존 가격 조회와 장기간 오프라인 기기의 복귀가 정상 동작하는지 확인한다.

개발 sandbox와 운영 데이터는 분리한다. 구독 중단·재연결과 API 제한·일시 오류는 미전송 요청을 잃지 않는 방식으로 처리한다. 반복 실패는 재시도 간격을 늘리되 사용자에게 대기·실패 상태와 재시도 방법을 보여 준다.

자동 확장이나 관리형 서비스라는 설명은 앱의 데이터 일관성·권한 설정·비용 제어를 자동으로 보장한다는 의미가 아니다.

11 구현 가능 여부와 검증 계획

기능	조사 결과	필수 추가 구현
사용자 구분	Cognito와 소유자 권한으로 구성 가능	함수 소유권 검사·로컬 계정 분리
목록 동기화	Gen 1 DataStore 또는 Gen 2 API 구성 가능	Gen 2 대기열·버전 검증·재조회
충돌과 중복	전략·조건 쓰기·요청 기록 활용 가능	후보 보존·선택 화면·장기 요청 결과
구매 확정	mutation과 DynamoDB 트랜잭션 구성 가능	부분 구매 분기·실패 복구·중복 판정
가격·공유	보조 인덱스와 사용자 query 구성 가능	공개 필드·공유 해제·반영 지연 처리

기능 확인 시험

동기화: 두 기기에서 같은 항목 수정, 수정·삭제, 전송 중 새 변경, 장기 오프라인 복귀를 시험한다. 충돌 선택 전 양쪽을 보존하고 선택 후 최종 상태가 일치해야 한다.

구매: 전체·부분·초과 구매, 로컬 실패, 서버 조건 실패, 저장 후 응답 유실과 동일 요청 재전송을 시험한다. 10분 이후 재전송도 중복 기록이 없어야 한다.

가격: 구매일·등록일 역전, 조건 불일치, 최신 기록 공유 해제, 인덱스 지연, 다른 계정 접근을 시험한다. 공개되지 않은 기록과 식별 정보가 노출되지 않아야 한다.

채택 판단에 필요한 결과

D-01~D-05의 환경·사용자 확인·충돌·구매·공개 정책을 확정하고 기능 시연과 실패 시험, 비용 견적을 남긴다. N-02의 변경 100건 동기화 10초와 N-04의 가격 조회 5초 등은 요구사항의 제안 목표이며 기기·네트워크 조건과 함께 측정한다.

AWS Amplify는 필요한 백엔드 구현 수단을 제공하지만 오프라인 동기화와 구매 일관성을 프로젝트 규칙에 맞게 설계해야 한다. Gen 2 구성은 API · 인증·서버 로직과 기기의 동기화 계층을 함께 검증하는 조건으로 판단한다. 이 조사만으로 채택이나 구현 완료를 확정하지 않는다.

12 공식 출처 인증과 동기화

2026년 10월 9일 확인. 본문 번호는 해당 공식 문서로 직접 연결된다. 제품·가격·SDK 조건은 채택 시점의 문서로 다시 확인한다.

[1] Amplify Flutter Gen 2 Quickstart
https://docs.amplify.aws/flutter/start/quickstart/

[2] Amplify Flutter Gen 2 Feature Parity
https://docs.amplify.aws/flutter/start/migrate-to-gen2/feature-matrix/

[3] Amplify DataStore에서 이전
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/migrate-from-datastore/

[4] Gen 1 DataStore 동작 구조
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/how-it-works/

[5] Gen 1 DataStore 충돌 처리
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/conflict-resolution/

[6] Gen 1 DataStore 이벤트
https://docs.amplify.aws/gen1/flutter/build-a-backend/more-features/datastore/datastore-events/

[7] Gen 2 인증 설정
https://docs.amplify.aws/flutter/build-a-backend/auth/set-up-auth/

[8] Gen 2 사용자 소유자 권한
https://docs.amplify.aws/flutter/build-a-backend/data/customize-authz/per-user-per-owner-data-access/

[9] Gen 2 모델 작업 비활성화
https://docs.amplify.aws/flutter/build-a-backend/data/data-modeling/disable-operations/

[10] Gen 2 사용자 정의 query와 mutation
https://docs.amplify.aws/flutter/build-a-backend/data/custom-business-logic/

[11] Gen 2 Lambda 함수 설정
https://docs.amplify.aws/flutter/build-a-backend/functions/set-up-function/

[12] Gen 2 함수의 자원 접근 권한
https://docs.amplify.aws/flutter/build-a-backend/functions/grant-access-to-other-resources/

[13] DynamoDB 트랜잭션 API
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html

13 공식 출처 조회와 비용

2026년 10월 9일 확인. 본문 번호는 해당 공식 문서로 직접 연결된다. 제품·가격·SDK 조건은 채택 시점의 문서로 다시 확인한다.

[14] AppSync DynamoDB 트랜잭션
https://docs.aws.amazon.com/appsync/latest/devguide/tutorial-dynamodb-transact.html

[15] Gen 2 보조 인덱스
https://docs.amplify.aws/flutter/build-a-backend/data/data-modeling/secondary-index/

[16] DynamoDB 글로벌 보조 인덱스
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html

[17] Flutter API 연결과 인증 모드
https://docs.amplify.aws/flutter/frontend/data/connect-to-API/

[18] Flutter 실시간 구독
https://docs.amplify.aws/flutter/frontend/data/subscribe-data/

[19] Flutter 사용자 정의 실시간 구독
https://docs.amplify.aws/flutter/frontend/data/custom-subscription/

[20] Flutter 로그아웃
https://docs.amplify.aws/flutter/frontend/auth/sign-out/

[21] AWS Amplify 과금 구조
https://aws.amazon.com/amplify/pricing/

[22] Amazon Cognito 요금
https://aws.amazon.com/cognito/pricing/

[23] AWS AppSync 요금
https://aws.amazon.com/appsync/pricing/

[24] Amazon DynamoDB 요금
https://aws.amazon.com/dynamodb/pricing/

[25] AWS Lambda 요금
https://aws.amazon.com/lambda/pricing/

[26] DynamoDB 읽기 일관성
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html